תשובות לשאלות מהתמונה (1-10)

1. מה עושה הפונקציה `fork()`? היא יוצרת תהליך חדש (Child) על ידי שכפול התהליך הנוכחי (Parent). שניהם ממשיכים לרוץ מהשורה שאחרי הקריאה לפונקציה.

2. מה ההבדל בין תהליך אב לתהליך בן? תהליך האב הוא המקור, ותהליך הבן הוא השכפול. ההבדל העיקרי הוא ב-PID (מספר הזיהוי) שלהם ובערך החזרה של `fork()`: האב מקבל את ה-PID של הבן, והבן מקבל 0.

3. מה עושה הפונקציה `wait()`? היא גורמת לתהליך האב להמתין (לעצור את הריצה) עד שאחד מתהליכי הבן שלו יסיים את ביצועו.

4. מה קורה אם לא נקרא ל-`wait()` אחרי `fork()`? נוצר מצב של "תהליך זומבי" (Zombie Process). הבן מסיים את ריצתו אך נשאר בטבלת התהליכים של המערכת כי האב לא "אסף" את סטטוס הסיום שלו.

5. מה ההבדל בין `fork()` ל-`pthread_create()`? `fork()` יוצר תהליך חדש עם מרחב זיכרון נפרד לחלוטין, בעוד `pthread_create()` יוצר Thread (חוט עבודה) חדש שחולק את אותו מרחב זיכרון עם שאר החוטים באותו תהליך.

6. מה עושה הפונקציה `pthread_join()`? היא המקבילה של `wait()` עבור Threads. היא גורמת ל-Thread שקרא לה להמתין עד ש-Thread ספציפי אחר יסיים את עבודתו.

7. מה ההבדל בין תהליך ל-Thread מבחינת זיכרון? תהליכים הם מבודדים – לכל אחד זיכרון משלו. Threads בתוך אותו תהליך חולקים את אותו קטע קוד, נתונים גלובליים וערימה (Heap), אך לכל אחד יש מחסנית (Stack) משלו.

8. מה עושה הספרייה `<unistd.h>`? זוהי ספריית הסטנדרט של מערכות POSIX (כמו לינוקס) המספקת גישה לשירותי מערכת ההפעלה, כמו `fork()`, `pipe` וניהול קבצים.

9. מה עושה הספרייה `<pthread.h>`? זוהי הספרייה המאפשרת עבודה עם Threads (חוטי עבודה) במערכות POSIX, וכוללת פונקציות ליצירה, סנכרון וניהול Threads.

10. מה ההבדל בין `return 0;` בתוך תהליך בן לבין בתוך ה-main? בתוך ה-`main`, פקודת `return 0;` מסיימת את כל התוכנית. בתוך תהליך בן (שנוצר ע"י `fork`), היא מסיימת רק את תהליך הבן ומחזירה את הסטטוס 0 לאב, בזמן שהאב ממשיך לרוץ.

חלק 2: מקביליות וסנכרון (Concurrency)

11. **איך ניתן להריץ שלוש פונקציות שונות במקביל עם `fork()`?** קוראים ל-`fork()` פעמיים. בכל פעם שהתנאי מזהה שאנחנו בתהליך הבן (`if (pid == 0)`), מריצים פונקציה אחת ויוצאים.
12. **איך ניתן להריץ שלוש פונקציות שונות במקביל עם Threads?** יוצרים שלושה Threads בעזרת `pthread_create()`, כאשר כל אחד מקבל מצביע לפונקציה אחרת כארגומנט.
13. **מה היתרון של שימוש ב-Threads על פני תהליכים?** הם "קלים" יותר (פחות Overhead למערכת), ויצירתם/החלפה ביניהם מהירה יותר. התקשורת ביניהם פשוטה כי הזיכרון משותף.

14. **מה היתרון של שימוש ב-`fork()` על פני Threads?** בטיחות ובידוד. אם תהליך אחד קורס, הוא לא מפיל את האחרים. אין סכנה של Race Condition על זיכרון משותף.
15. **מה ההבדל בין שימוש ב-`sleep()` בתהליך לבין ב-Thread?** ב-`fork`, רק התהליך שקרא ל-`sleep` נעצר. ב-Threads, הפונקציה `sleep` עלולה לעצור את כל התהליך (תלוי במימוש), לכן משתמשים בגרסאות ייעודיות ל-Threads.
16. **מה קורה אם שני Threads ניגשים לאותו משתנה גלובלי?** עלול להיווצר מצב של אי-עקביות בנתונים (Race Condition) אם לפחות אחד מהם כותב למשתנה.
17. **מה זה מצב של Race Condition?** מצב שבו תוצאת התוכנית תלויה בתזמון הלא צפוי של הרצת ה-Threads. זה קורה כששניהם מנסים לשנות את אותו משאב בו-זמנית. 18. **איך ניתן למנוע מצב של Race Condition?** משתמשים במנגנוני סנכרון כמו **Mutex** (מנעולים) או Semaphores כדי להבטיח שרק Thread אחד יגש למשאב הקריטי בכל רגע.
18. **מה ההבדל בין משתנה `static` לבין משתנה גלובלי?** משתנה גלובלי נגיש לכל הקבצים בתוכנית. משתנה `static` (מחוץ לפונקציה) נגיש רק בתוך הקובץ שבו הוא הוגדר.
19. **מה ההבדל בין `pthread_join` ל-`pthread_create` מבחינת תפקיד?** `create` הוא ה"מתניע" (יוצר את החוט ומתחיל להריץ), ו-`join` הוא ה"ממתין" (מוודא שהחוט סיים לפני שממשיכים).

התמונות שהעלית מצוינות ומכסות את כל הבסיס של ניהול תהליכים וחוטי עבודה (Threads) במערכות הפעלה. ריכזתי עבורך את כל 30 השאלות מהתמונות עם תשובות קצרות וקולעות:

חלק 1: יסודות ה-Processes (שאלות 1-10)

1. **מה עושה הפונקציה `fork()`?** יוצרת תהליך חדש על ידי שכפול התהליך הקיים.
2. **מה ההבדל בין תהליך אב לתהליך בן?** האב הוא המקור, הבן הוא העותק. ההבדל הוא ב-PID ובערך החזרה של ה-fork (אב מקבל את ה-PID של הבן, בן מקבל 0).
3. **מה עושה הפונקציה `wait()`?** גורמת לאב להמתין עד שתהליך בן יסיים את ריצתו.
4. **מה קורה אם לא נקרא ל-`wait()` אחרי `fork()`?** תהליך הבן הופך ל"זומבי" (Zombie) – הוא סיים אך עדיין רשום במערכת.
5. **מה ההבדל בין `fork()` ל-`pthread_create()`?** הראשון יוצר תהליך נפרד (זיכרון מבודד), השני יוצר Thread (זיכרון משותף).
6. **מה עושה הפונקציה `pthread_join()`?** גורמת ל-Thread להמתין לסיום של Thread אחר.
7. **מה ההבדל בין תהליך ל-Thread מבחינת זיכרון?** תהליכים מבודדים; Threads חולקים את אותו מרחב כתובות (Stack נפרד לכל אחד, Heap וגלובליים משותפים).
8. **מה עושה הספרייה `<unistd.h>`?** מספקת גישה לשירותי מערכת ההפעלה (כמו `fork`, `pipe`, `sleep`).
9. **מה עושה הספרייה `<pthread.h>`?** מאפשרת עבודה עם Threads במערכות POSIX.
10. **מה ההבדל בין `return 0;` בתוך תהליך בן לבין בתוך ה-main?** ב-main זה מסיים את התוכנית כולה; בבן זה מסיים רק את הבן ומחזיר סטטוס לאב.

חלק 2: מקביליות וסנכרון (שאלות 11-20)

11. **איך ניתן להריץ שלוש פונקציות במקביל עם `fork()`?** מבצעים `fork` פעמיים בתוך לולאה או מבנה תנאי, כך שכל בן מריץ פונקציה אחרת.
12. **איך ניתן להריץ שלוש פונקציות במקביל עם Threads?** קוראים ל-`pthread_create` שלוש פעמים, כל פעם עם מצביע לפונקציה אחרת.
13. **מה היתרון של שימוש ב-Threads על פני תהליכים?** יצירה מהירה יותר ותקשורת קלה בזכות זיכרון משותף.
14. **מה היתרון של שימוש ב-`fork()` על פני Threads?** בידוד ובטיחות – אם תהליך אחד קורס, הוא לא מפיל את האחרים.
15. **מה ההבדל בין שימוש ב-`sleep()` בתהליך לבין ב-Thread?** בתהליך זה עוצר את כל התהליך. ב-Threads יש לוודא שהפונקציה אינה חוסמת את כל שאר ה-Threads (תלוי במימוש ה-Library).
16. **מה קורה אם שני Threads ניגשים לאותו משתנה גלובלי?** עלולה להיווצר השחתת נתונים אם שניהם כותבים בו-זמנית.
17. **מה זה מצב של race condition?** מצב שבו תוצאת התוכנית תלויה בסדר הרצה לא צפוי של תהליכים/Threads.
18. **איך ניתן למנוע מצב של race condition?** שימוש במנגנוני סנכרון כמו Mutex או Semaphore.
19. **מה ההבדל בין משתנה `static` למשתנה גלובלי?** גלובלי נגיש לכל הקבצים; `static` נגיש רק בתוך הקובץ בו הוגדר.
20. **מה ההבדל בין `pthread_join` ל-`pthread_create` מבחינת תפקיד?** `create` מתניע את ה-Thread; `join` מחכה שהוא יסיים.

חלק 3: תקשורת וסנכרון מתקדם (שאלות 21-30)

21. **איך ניתן לסנכרן Threads באמצעות `mutex`?** נועלים את המנעול לפני הכניסה לקטע הקריטי ומשחררים מיד בסיום.
22. **איך ניתן לסנכרן Threads באמצעות `semaphore`?** משתמשים במונה המאפשר למספר מוגדר של Threads להיכנס לקטע קוד.
23. **מה ההבדל בין `mutex` ל-`semaphore`?** מוטקס הוא מנעול בינארי (0/1); סמפור הוא מונה שיכול לאפשר ליותר מ-Thread אחד גישה.
24. **איך ניתן לשתף מידע בין תהליכים שנוצרו עם `fork()`?** דרך IPC (Pipes, Shared Memory, Sockets).
25. **מה זה shared memory?** קטע זיכרון ששני תהליכים שונים יכולים לקרוא ולכתוב אליו במקביל.
26. **איך משתמשים ב-`pipes` לתקשורת בין תהליכים?** יוצרים צינור שבו צד אחד כותב (`write`) והצד השני קורא (`read`).
27. **מה ההבדל בין תקשורת בין תהליכים (IPC) לבין שיתוף משתנים ב-Threads?**
ב-Threads זה פשוט גישה לאותה כתובת זיכרון; ב-IPC נדרש תיווך של מערכת ההפעלה.
28. **איך ניתן לגרום ל-Thread להפסיק פעולתו באמצעות `flag`?** ה-Thread בודק משתנה בוליאני בלולאה; כשהאב משנה את ה-Flag, ה-Thread מסיים את הלולאה ויוצא.

29. **איך ניתן לגרום לתהליך להפסיק פעולתו באמצעות `signal`?** שולחים סיגנל (כמו SIGKILL או SIGTERM) באמצעות פקודת `kill`.
30. **מה ההבדל בין `return` לבין `()exit` בתוך תהליך?** `return` יוצא מהפונקציה הנוכחית; `()exit` סוגר את התהליך כולו מיד.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/wait.h>

int shared_var = 10; // משתנה גלובלי

// פונקציה עבור ה-Thread
void* thread_function(void* arg) {
    shared_var += 5;
    printf("Thread: שיניתי את המשתנה ל-%d\n", shared_var);
    return NULL;
}

int main() {
    printf("--- בדיקת Threads ---\n");
    pthread_t my_thread;
    pthread_create(&my_thread, NULL, thread_function, NULL);
    pthread_join(my_thread, NULL);
    printf("Main (Thread Test): הערך אחרי ה-Thread הוא %d (השתנה!)\n\n", shared_var);

    shared_var = 10; // איפוס הערך לבדיקה הבאה

    printf("--- בדיקת Fork ---\n");
    pid_t pid = fork();

    if (pid == 0) { // תהליך הבן
        shared_var += 5;
        printf("Child Process: שיניתי את המשתנה ל-%d\n", shared_var);
        exit(0);
    } else { // תהליך האב
        wait(NULL); // מחכה שהבן יסיים
        printf("Parent Process: הערך אצלי הוא עדיין %d (לא השתנה!)\n", shared_var);
    }

    return 0;
}
```

מה קרה כאן? ההסבר הטכני

1. **ב-Threads:** כאשר ה-Thread שינה את `shared_var`, השינוי קרה בכתובת זיכרון שמשותפת לכל התוכנית. לכן ה-Main ראה את הערך המעודכן (15).
2. **ב-Fork:** ברגע ה-`fork`, מערכת ההפעלה יצרה **עותק** של כל הזיכרון. למרות שלשני התהליכים יש משתנה בשם `shared_var`, הם יושבים במקומות פיזיים שונים בזיכרון. הבן שינה את העותק שלו, אבל הזיכרון של האב נשאר "נקי" (10).

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define ROWS 3
#define COLS 3

// הגדרת המטריצה
int matrix[ROWS][COLS] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// פונקציה שכל Thread יריץ
void* print_row(void* arg) {
    int row_index = *((int*)arg); // קבלת מספר השורה

    printf("Thread ID %lu משימה: מדפיס שורה %d: ", (unsigned long)pthread_self(), row_index);

    for (int j = 0; j < COLS; j++) {
        printf("%d ", matrix[row_index][j]);
    }
    printf("\n");

    free(arg); // שחרור הזיכרון שהוקצה למספר השורה
    return NULL;
}

int main() {
    pthread_t threads[ROWS];

    printf("מתחיל הדפסת מערך דו-ממדי בעזרת Threads:\n\n");

    for (int i = 0; i < ROWS; i++) {
        // בבטחה Thread-הקצאת זיכרון כדי להעביר את מספר השורה ל
```

```
        int* row_to_print = malloc(sizeof(int));
        *row_to_print = i;

        // יצירת Thread לכל שורה
        if (pthread_create(&threads[i], NULL, print_row, row_to_print) != 0) {
            perror("נכשל ביצירת Thread");
            return 1;
        }
    }

    // המתנה לכל ה-Threads (סנכרון שיסיימו)
    for (int i = 0; i < ROWS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\nכל ה-Threads סיימו את עבודתם.\n");
    return 0;
}
//////////////////
```

חלק 4: ניהול וביצועים (שאלות 31–40)

31. **איך ניתן ליצור Thread pool?** יוצרים מספר קבוע של Threads מראש (בזמן עליית התוכנית) ומכניסים אותם לתור המתנה. בכל פעם שיש משימה חדשה, אחד ה-Threads הפנויים מבצע אותה במקום ליצור Thread חדש.
32. **מה היתרון של Thread pool על פני יצירת Threads חדשים בכל פעם?** חיסכון בזמן ה"תקורה" (Overhead) של יצירת והריסת Threads, ומניעת מצב של הצפת המערכת ביותר מדי Threads ממה שהיא יכולה להכיל.
33. **איך ניתן למדוד זמן ריצה של תהליך?** משתמשים בפונקציות כמו `clock()` או בבודק המערכת החיצוני `time` (בשורת הפקודה).
34. **איך ניתן למדוד זמן ריצה של Thread?** משתמשים בפונקציות זמן ייעודיות כמו `clock_gettime()` עם הדגל `CLOCK_THREAD_CPUTIME_ID`.
35. **איך ההבדל בין תזמון תהליכים לתזמון Threads?** תהליכים מתוזמנים על ידי מערכת ההפעלה (OS scheduling). Threads יכולים להיות מתוזמנים על ידי המערכת (Kernel threads) או על ידי ספריית המשתמש (User threads).
36. **איך מערכת ההפעלה מחליטה איזה תהליך ירוץ?** באמצעות מתזמן (Scheduler) המשתמש באלגוריתמים כמו Round Robin (תור), Priority (עדיפות) או Shortest Job First.
37. **איך מערכת ההפעלה מחליטה איזה Thread ירוץ?** בדומה לתהליכים, אך במערכות מודרניות ה-Scheduler מתייחס לכל Thread כיחידת ריצה עצמאית (LWP - Light Weight Process).
38. **מה זה context switch?** התהליך שבו המעבד שומר את המצב של תהליך/Thread נוכחי (Registers, PC) וטוען את המצב של הבא בתור כדי להחליף ביניהם.
39. **מה ההבדל בין context switch של תהליך לבין של Thread?** החלפת Threads באותו תהליך מהירה בהרבה כי אין צורך להחליף את מרחב הזיכרון (Memory space) וה-Cache לא נמחק לגמרי.

40. **איך ניתן להשתמש ב-`pthread_mutex_lock` ו-`pthread_mutex_unlock`?** קוראים ל-`lock` לפני הקטע הקריטי (שינוי משתנה משותף) כדי למנוע מאחרים להיכנס, ול-`unlock` מיד בסיום הקטע כדי לשחרר את הגישה.

חלק 5: בעיות מורכבות ופתרונות (שאלות 41–50)

41. **איך ניתן לממש מערכת שבה תהליכים שונים עובדים על חלקים שונים של קובץ?** משתמשים ב-`fork` וכל תהליך מקבל "אופסט" (Offset) שונה בתוך הקובץ בעזרת `lseek`.
42. **איך ניתן לממש מערכת שבה Threads שונים עובדים על חלקים שונים של מערך?** מחלקים את אינדקס המערך למספר ה-Threads, וכל Thread מקבל טווח אינדקסים ספציפי (למשל: Thread 1 על תאים 0-99).
43. **איך ניתן למנוע deadlock בין Threads?** הקפדה על נעילת Mutex-ים בסדר קבוע תמיד, או שימוש ב-`pthread_mutex_trylock` שאינו חוסם את הריצה.
44. **מה זה deadlock וכיצד הוא נגרם?** מצב שבו Thread א' מחכה למנעול שמוחזק ע"י Thread ב', בזמן ש-Thread ב' מחכה למנעול שמוחזק ע"י א'. שניהם תקועים לנצח.
45. **איך ניתן לזהות מצב של deadlock?** בעזרת כלים כמו `valgrind --tool=helgrind` או בניית גרף תלויות בתוכנה ובדיקה אם יש בו "מעגל" (Cycle).
46. **איך ניתן למנוע starvation בתזמון Threads?** שימוש באלגוריתמי תזמון הוגנים (Fair scheduling) והימנעות ממתן עדיפות גבוהה מדי ל-Threads מסוימים לאורך זמן.
47. **איך ניתן לממש producer-consumer באמצעות Threads?** משתמשים בחוצץ משותף (Buffer), Mutex להגנה על התור, ומשתני תנאי (`pthread_cond_t`) כדי להודיע כשיש מוצר חדש או מקום פנוי.
48. **איך ניתן לממש producer-consumer באמצעות תהליכים?** דומה ל-Threads, אך משתמשים בזיכרון משותף (Shared Memory) ובסמפורים (Semaphores) של המערכת לסנכרון.
49. **איך ניתן לממש מערכת שבה תהליכים מתקשרים באמצעות message queue?** משתמשים ב-API של מערכת ההפעלה (כמו `msgsnd` ו-`msgrcv`) המאפשר שליחת הודעות מובנות לתוך תור בניהול ה-Kernel.
50. **איך ניתן לממש מערכת שבה Threads מתקשרים באמצעות משתנים משותפים ומנגנוני סנכרון?** מגדירים משתנים גלובליים ומשתמשים ב-Mutex כדי להבטיח שרק אחד כותב/קורא בכל רגע, וב-Condition Variables כדי לאותת על שינויים.

רמה בסיסית (1–10)

1. **כתוב סקריפט שמדפיס "שלום עולם":** `echo "שלום עולם"`
2. **סקריפט שמקבל שם מהמשתמש ומדפיס "שלום <שם>":** משתמשים ב-`read name` ואז `echo "שלום $name"`.
3. **סקריפט שמקבל מספר ומדפיס אם הוא גדול מ-10 או לא:** משתמשים במבנה `if [ $num -gt 10 ]`.

4. **סקריפט שמדפיס את כל המספרים מ-1 עד 20:** לולאת `for i in {1..20}; do echo $i; done`.
5. **סקריפט שמחשב סכום של שני מספרים שלמים:** `echo $((num1 + num2))`.
6. **סקריפט שמחשב סכום של שני מספרים עשרוניים (עם bc):** `echo "$num1 + $num2" | bc`.
7. **סקריפט שמדפיס את שם התיקייה הנוכחית (pwd):** פשוט מריצים את הפקודה `pwd`.
8. **סקריפט שמדפיס את כל הקבצים בתיקייה הנוכחית (ls):** פשוט מריצים את הפקודה `ls`.
9. **סקריפט שסופר כמה קבצים יש בתיקייה:** `ls | wc -l`.
10. **סקריפט שמבקש מהמשתמש מספר ומדפיס את לוח הכפל שלו:** לולאת `for` מ-1 עד 10 שמכפילה את המשתנה במיקום הלולאה.

רמה בינונית (11–20)

11. **סקריפט שמקבל מספר ומדפיס אם הוא זוגי או אי-זוגי:** בודקים שארית חלוקה: `if [ $((num % 2)) -eq 0 ]`.
12. **סקריפט שמדפיס את כל הקבצים עם סיומת sh. בתיקייה:** `ls *.sh`.
13. **סקריפט שמעתיק קובץ אחד לתיקייה אחרת:** `cp $file $destination`.
14. **סקריפט שמוחק קובץ אם הוא קיים:** בדיקת תנאי: `if [ -f $file ]; then rm $file; fi`.
15. **סקריפט שמבקש מהמשתמש שם קובץ ומדפיס את מספר השורות בו (wc -l):** `wc -l < $filename`.
16. **סקריפט שמבקש מהמשתמש מחרוזת ומחפש אותה בקובץ (grep):** `grep "$string" $filename`.
17. **סקריפט שמדפיס את התאריך והשעה הנוכחיים (date):** פשוט מריצים את הפקודה `date`.
18. **סקריפט שמדפיס את כל משתני הסביבה (env):** פשוט מריצים את הפקודה `env`.
19. **סקריפט שמבקש סיסמה ומדפיס "גישה מותרת" אם היא נכונה:** מבנה `if [ "$input" == "secret" ]`.
20. **סקריפט שמבצע לולאה ומדפיס את כל המספרים מ-100 עד 1 בסדר יורד:** `for i in {100..1}; do echo $i; done`.

בונוס: איך נראה קוד כזה בפועל?

כדי לענות על השאלות האלו במבחן או בשיעורי בית, כך נראה מבנה של סקריפט Bash טיפוסי (למשל עבור שאלה 11):

Bashi